01

AI-POWERED AUTOMATIC SOFTWARE TESTING

01

WHY TRADITIONAL TESTING FAILS

The Shift from Scripted Verification to Intelligent Validation

02

The Fragility Problem

- Scripts break with minor UI changes.
- High cost of updating locators.
- Maintenance burden exceeds development time.

03

The UX Gap

- Verifies "if it works" but ignores usability.
- Technically clickable != Visually accessible.
- Blind to layout shifts and visual bugs.

04



Fragile Selectors: The #1 cause of CI/CD pipeline failures.

01

CURRENT AI TESTING LANDSCAPE

Augmenting Automation with Intelligence

02

AI Test Generation

Converting natural language instructions into executable Playwright/Selenium scripts.

Natural Language Processing

Self-Healing

Automatically detecting UI changes and repairing broken selectors in real-time.

Dynamic Element Discovery

03

Visual Regression

Using Computer Vision to detect UI defects, layout shifts, and visual inconsistencies.

Vision Language Models

Contextual Analysis

Natural language explanations of failures instead of cryptic log files.

Semantic Debugging

04



"The integration of VLMs reduces script maintenance by up to 70%."

01

MULTI-MODAL INPUTS

Orchestrating Data for Autonomous Execution

02

Requirement Sources

- Natural Language Instructions
- Product Requirement Docs (PRDs)
- Technical Specifications

03

Process Discovery

- Manual Step-by-Step Procedures
- Existing Test Files (Legacy)
- User Interaction Logs

04



DATA CONVERGENCE ENGINE

All inputs feed into a centralized AI testing engine for intent-based generation.

01

GOAL-ORIENTED TESTING

Beyond Rigid Scripted Execution

02

TRADITIONAL APPROACH

FIXED SEQUENCES:

1. Click Element #ID_42
2. Input "Admin" to #User
3. Click Button .submit-btn

- Breaks on layout changes.
- Requires exact selectors.
- No contextual awareness.

03

AI-DRIVEN APPROACH

HIGH-LEVEL GOAL:

"Purchase a laptop using a guest account."

- Dynamic path discovery.
- Understands intent & UI.
- Resilient to refactors.

04

ADVANCED CAPABILITIES

- **Persona Simulation:** Test as "Frustrated User" or "Power User".
- **UX Evaluation:** Detect friction points in real-time.
- **Self-Correction:** Recovers from unexpected pop-ups.

01

VISION-DRIVEN ARCHITECTURE

Mimicking Human Perception for Testing

02

The Vision Loop

- Application State
- Screenshot Capture
- VLM Processing
- UI Identification
- Screen Coordinates

03

Key Advantages

Perceptual Accuracy

No dependency on DOM or fixed selectors. AI sees what the user sees.

Extreme Resilience

Robust against code refactors, ID changes, and framework migrations.

04

AI Perception Engine

07

ENHANCED MULTI-SOURCE VISION

Fusing Visual and Structural Intelligence

08

Input Fusion Sources

- Screenshots (Visual)
- DOM Tree (Structural)
- Accessibility Tree
- OCR Data (Textual)
- Network Logs (State)

09

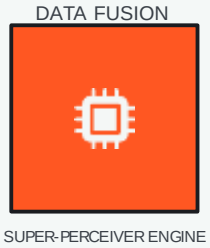
99%+ Perception Accuracy

Cross-referencing visual data with structural metadata eliminates false positives.

Contextual Depth

Memory and logs provide a deeper understanding of application state transitions.

10



08

EXECUTION STRATEGIES

Machine vs. Human Interaction

09

Path A: Hybrid Execution

VLM Perception + Playwright/DOM

- >> High speed and precision.
- >> Native browser integration.
- >> Best for web-based SaaS.
- >> Reliable element targeting.

10

Path B: Digital Human

OS-Level Mouse & Keyboard

- >> Behaves exactly like a user.
- >> Bypasses browser sandboxes.
- >> Zero dependency on DOM/Code.
- >> True end-to-end validation.

11

CROSS-PLATFORM VERSATILITY

Browsers | Desktop Apps | SolidWorks | VS Code | Legacy Software

09

DIGITAL HUMAN CAPABILITIES

Autonomous Agents for Any Graphical Interface

10

Core Execution

- >> Complete multi-step user journeys.
- >> Persona-based simulation (e.g., Power User).
- >> Cross-platform interaction (Web, Desktop).
- >> Real-world mouse & keyboard emulation.

11

Analytical Intelligence

- >> Real-time UX friction detection.
- >> Adaptive learning for UI refactors.
- >> Visual regression & layout validation.
- >> Intent-based goal navigation.

12

Output & Reporting

- >> Intelligent, human-readable bug reports.
- >> Natural language failure explanations.
- >> Video/Screenshot evidence generation.
- >> Performance & latency benchmarking.

10

PERFORMANCE OPTIMIZATION

Efficiency in VLM Workflows

11

The VLM Challenge

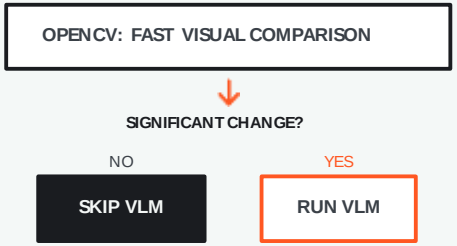
>> Direct screenshot comparison is slow.

>> High API costs for continuous polling.

>> Redundant processing of static screens.

12

The OpenCV Solution



13

Efficiency Gains

>> **Speed:** Up to 10x faster execution.

>> **Cost:** 80% reduction in API usage.

>> **Scale:** Supports high-frequency testing.

11

AUTONOMOUS ARCHITECTURE

The Continuous Intelligent Testing Loop

12

Core Loop Components

- Planner (LLM)
- Perception (Vision/OCR)
- Decision Engine
- Actuator (Real Interaction)

13

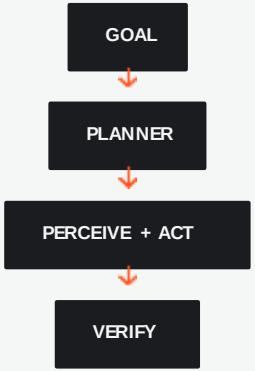
Operational Logic

The Planner breaks down high-level goals into actionable steps while maintaining a persistent memory of previous states.

Goal Achievement

The loop iterates continuously, adapting to screen changes until the objective is met or a bug is identified.

14



THE FUTURE VISION

Towards Autonomous AI Test Engineers

Next-Gen Capabilities

>> Autonomous Discovery: AI that explores apps to find bugs.

>> Self-Generating Test Suites based on PRDs.

>> Deep Architectural & UX Feedback.

>> Real-time Adaptation to UI Refactors.

KEY TAKEAWAY:

Move beyond script-based automation toward intelligent, autonomous, human-like software testing powered by LLMs and VLMs.